

Change Report — ADF Analyzer (DataFlows Changes)

Date: 2025-12-30

Author: Development assistant

Executive Summary

This report documents a narrowly scoped change to the ADF Analyzer that adds a lightweight complexity metric to DataFlows exports and documents the new fields. The change is limited to computing a weighted transformation score per DataFlow, assigning a complexity bucket, and exposing both fields in the exported **DataFlows** sheet and in the **DataDictionary**.

Key Changes (DataFlows only)

- File: **ADF_Analyzer_v10_Production/core/adf_analyzer_v10_complete.py**
 - Added a weighted transformation score and complexity bucket to **DataFlows** records.
 - New fields added to each dataflow record: **TransformationScore** (integer) and **TransformationComplexity** (text: **Low**/**Medium**/**High**).
 - Implementation summary:
 - Transformation counts are discovered during script parsing and aggregated per dataflow.
 - A weight map is applied to recognized transformation types and the score is computed as the sum(count × weight).
 - Complexity buckets: Score ≤ 5 → **Low**; 6–10 → **Medium**; > 10 → **High**.
 - Weights used (per design): Source/Sink=1, DerivedColumn=2, Filter=1, Join=4, Lookup=4, Aggregate=5, ConditionalSplit=4, Exists/Assert=5, Union=3.
 - Integrated the two new fields into the dynamic **DataFlows** placeholder schema so placeholder sheets and exported files include the columns consistently.
 - Updated the in-module **DataDictionary** so the workbook documents **TransformationScore** and **TransformationComplexity**.

Files Modified

- **ADF_Analyzer_v10_Production/core/adf_analyzer_v10_complete.py**
 - Added computation of **TransformationScore** and **TransformationComplexity** when building each **dataflow_rec**.
 - Added **TransformationScore** and **TransformationComplexity** to the DataFlows placeholder schema.
 - Added **DataDictionary** entries documenting the new fields.

Rationale

- Provide a quick, interpretable indicator of DataFlow complexity to help triage maintenance and performance prioritization.
- Keep the change lightweight and deterministic so it is easy to review and revert if needed.

How to Verify (manual steps)

1. Run the analyzer locally using the existing runner command in the repository root:

```
python .\core\adf_analyzer_v10_patched_runner.py temp_test2.json
```

2. Open the produced Excel workbook and verify:

- **DataFlows** sheet: contains **TransformationScore** and **TransformationComplexity** columns with values for each dataflow.
- **DataDictionary** sheet: documents **TransformationScore** and **TransformationComplexity**.

3. Spot-check a dataflow with multiple transformation types and confirm the score equals the sum of counts × weights for recognized types.

Example (snippet)

```
tf_weights = {'Source':1, 'Sink':1, 'DerivedColumn':2, 'Filter':1, 'Join':4,
...}
score = sum(transformation_counts.get(t,0) * tf_weights.get(t,0) for t in
transformation_counts)
complexity = 'Low' if score <=5 else 'Medium' if score <=10 else 'High'
```

Risks & Mitigations

- Risk: Weight mapping may omit aliased/unrecognized transformation names and thus undercount.
 - Mitigation: Unrecognized types default to weight 0; weights can be extended or made configurable.

Suggested Next Steps

- (Optional) Add unit/integration tests that assert expected **TransformationScore** values for synthetic dataflows.
- (Optional) Make weights configurable or add aliases mapping for transformation names.

Location of the Report

Saved to: **ADF_Analyzer_v10_Production/CHANGE_REPORT_adf_analyzer_changes.md**

If you want, I can run the analyzer now and attach the generated Excel file showing the new DataFlows columns, or add a unit test to validate the score calculation.

Which would you like next?